Pandas DataFrames - Creation, Indexing & Selection

Suman 15 Feb, 2026 At 8:00 PM Pre-Reads Module-1 Recommended Resource

Associated Lectures (1)

Description Discussions

Session 4.1: Pre-read Material

Pandas DataFrames: Creation, Indexing & Selection

Program: AIM101 - Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna x Masai School **Session Duration:** 105 minutes **Preparation Time:** 20-30 minutes

---

What This Session Covers

This session marks the start of **Week 4** and your formal introduction to **Pandas** -- the single most important Python library for working with real-world tabular data. You will transition from NumPy (numerical arrays accessed by integer positions) to Pandas (labeled tables accessed by meaningful names). The session covers eight topics:

Why Pandas? -- Excel Horror Stories that show what goes wrong without the right tool, plus a clear NumPy vs Pandas comparison

Series -- The 1D labeled array (think: a single spreadsheet column with row names)

DataFrame Creation -- Building tables from dictionaries, lists of lists, NumPy arrays, and files

Reading External Data -- Loading CSV files with `pd.read_csv()` and exploring your data with `.head()`, `.info()`, `.describe()`, `.shape`

Column Selection -- Picking columns using bracket notation and dot notation

Row Selection -- Accessing rows by label (`.loc`) and by position (`.iloc`) -- the most important distinction in Pandas

Boolean Filtering -- Filtering rows with conditions and the `.query()` method

Sorting -- Ordering your DataFrame with `.sort_values()` and `.sort_index()`

By the end of this session, you will be able to load a CSV file, select exactly the rows and columns you need, filter with conditions, and sort the results -- the fundamental operations behind every data analysis workflow.

Prerequisites Checklist

Before attending this session, make sure you are comfortable with:

- [ ] Creating NumPy arrays using `np.array()`, `np.zeros()`, `np.arange()`
- [ ] Element-wise operations on arrays (adding, multiplying, comparing)
- [ ] Boolean masking: `arr[arr > 0]` to filter elements that meet a condition
- [ ] Fancy indexing: `arr[[0, 2, 4]]` to select specific positions
- [ ] Reshaping arrays and understanding `.shape`
- [ ] Using `np.sum()`, `np.mean()`, `np.max()` for aggregations
- [ ] All Python fundamentals: variables, loops, if/elif/else, lists, tuples, dictionaries, sets, functions

If any of these feel unclear, review your Session 3.1 and 3.2 materials before class.

Important note: Pandas is started completely from scratch in this session. No prior Pandas knowledge is assumed or required. If you are comfortable with NumPy, you are ready.

Business Story: Excel Horror Stories

Before you write a single line of Pandas code, you need to understand WHY this library exists. The answer is not academic -- it comes from real disasters that cost real money and wasted real years of research.

The Gene Name Massacre

Scientists discovered that Microsoft Excel had been **silently corrupting gene names** in genomics research for years. Gene name `SEPT2` (Septin-2, a protein involved in cell division) was auto-converted to **"September 2"**. Gene name `MARCH1` (a protein critical in immune response) became **"March 1"**. Gene name `DEC1` turned into **"December 1"**. Excel simply decided these text strings looked like dates and converted them -- silently, with no warning.

```
BEFORE (in CSV):        AFTER (opened in Excel):
SEPT2                   02-Sep
MARCH1                  01-Mar
DEC1                    01-Dec
2310009E13              2.31E+13
```

A 2020 study in *PLOS Computational Biology* found that approximately **30% of published genomics papers** with supplementary Excel files contained corrupted gene data. The problem was so severe that the Human Gene Nomenclature Committee actually **renamed human genes** to avoid Excel corruption (SEPT2 became SEPTIN2, MARCH1 became MARCHF1). Biology had to adapt to software because the software would not adapt to biology.

The $6.2 Billion Copy-Paste

In 2012, JPMorgan Chase suffered approximately **$6.2 billion in trading losses** in the "London Whale" incident. A critical contributing factor was an Excel-based Value-at-Risk (VaR) model. A formula was supposed to divide by the **average** of two numbers but instead divided by their **sum** -- a copy-paste error

in a VBA macro that made reported risk appear roughly half of the actual risk. There was no version control, no automated testing, no code review. Errors compounded silently.

```
EXCEL PIPELINE:                    PANDAS PIPELINE:
 Manual data entry                  pd.read_csv()
 Copy-paste formulas                Scripted transforms
 VBA macros                         Python functions
 Email spreadsheet                  Git version control
 Hope nothing broke                 Automated tests
```

Why This Matters for You

Every one of these disasters shares a common thread: professionals used a tool designed for small-scale manual work on problems that demanded programmatic, reproducible, scalable solutions. Pandas prevents all of these failure modes -- no auto-formatting, no row limits (beyond available RAM), full audit trail through code, and complete reproducibility.

Think About:

- Have you ever had Excel change your data unexpectedly?
- If you build a financial model in a spreadsheet, how would a colleague verify your formulas are correct?
- Why might "reproducibility" matter in scientific research?

Key Concepts to Preview

Read through these previews carefully. You do not need to memorise every detail -- the goal is to arrive in class with a general sense of what each concept is, so the hands-on demonstrations feel familiar rather than foreign.

What Is Tabular Data?

Before we talk about Pandas, we need to talk about the kind of data it is designed for: **tabular data** -- data organised in rows and columns, like a table. You have seen tabular data your entire life: a class attendance register, a grocery bill, a cricket scorecard.

Name	Age	City	Salary
Alice	28	Mumbai	75000
Bob	34	Delhi	62000
Charlie	22	Bangalore	58000
Diana	31	Chennai	81000

The rules are straightforward:

- **Each row** represents one record (one person, one transaction, one observation)

- **Each column** represents one attribute (name, age, city, salary)
- **Every value in a column** is the same type of thing (the Age column is always a number, the City column is always text)

This is the most common format for business, scientific, and government data. When someone says "I have a dataset," they almost always mean a table like this.

NumPy vs Pandas: Different Tools for Different Jobs

You spent all of Week 3 learning NumPy. It is a powerful library. So why learn another one?

NumPy is designed for **numerical computation** -- mathematical operations on arrays of numbers where every element is the same type. Pandas is designed for **tabular data analysis** -- datasets where columns have names, rows have identifiers, and different columns can have different types (text in one, numbers in another, dates in a third).

Feature	NumPy	Pandas
Data type	Homogeneous (all same type)	Heterogeneous (mixed types per column)
Labels	Integer positions only	Named columns and named rows
Best for	Numerical computation, linear algebra	Tabular data, data cleaning, analysis
Missing data	No native support	Built-in NaN handling
File I/O	Limited (.npy, .txt)	CSV, Excel, SQL, JSON, HTML, and more

Key insight: NumPy is the **engine** under the hood; Pandas is the **car** you drive. Pandas uses NumPy internally for all its numerical operations. You are not replacing NumPy -- you are adding a user-friendly interface on top of it for tabular data tasks. You will use both in every data science project.

Series: A Column With a Name Tag

A **Series** is the simplest Pandas data structure: a **1D array where every value has a label**. Think of it as one column torn out of a spreadsheet, but with the row headers still attached.

```
import pandas as pd

# Create a Series from a dictionary
scores = pd.Series({'Alice': 85, 'Bob': 92, 'Charlie': 78, 'Diana': 95})
print(scores)
```

Output:

```
Alice      85
Bob        92
Charlie    78
```

```
Diana      95
dtype: int64
```

Notice how each value has a meaningful label (Alice, Bob, ...) instead of just a position number. Under the hood, the values are stored as a NumPy array -- all your NumPy skills still apply.

Think About:

- How is a Series different from a Python dictionary?
- What advantage does a labeled index give you over a plain NumPy array?

DataFrame: The Entire Table

A **DataFrame** is the star of Pandas -- a **2D table with named rows and named columns**. If a Series is one column, a DataFrame is the entire spreadsheet.

```
import pandas as pd

employees = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'age': [28, 35, 42, 31],
    'salary': [55000, 72000, 68000, 61000],
    'department': ['Engineering', 'Sales', 'Engineering', 'Marketing']
})
print(employees)
```

Output:

```
      name  age  salary   department
0    Alice   28   55000  Engineering
1      Bob   35   72000        Sales
2  Charlie   42   68000  Engineering
3    Diana   31   61000    Marketing
```

Each column can hold a different data type (text, integers, floats) and each column is actually a Series that shares the same row index with all other columns.

Think About:

- What would it take to represent this same data using only NumPy arrays? How many arrays would you need?
- Why is `employees['department']` easier to read than `data[:, 3]`?

Loading Data: pd.read_csv()

In practice, you will rarely type your data by hand. The most common way to get data into Pandas is from a CSV file -- a plain text file where values are separated by commas.

```
import pandas as pd
```

```
df = pd.read_csv('orders.csv')
print(df.head())  # Shows the first 5 rows
```

One line of code loads an entire dataset into a DataFrame, complete with column headers and proper data types. You will learn the essential exploration methods (`.head()` , `.info()` , `.describe()` , `.shape`) that serve as a "health check" every time you encounter a new dataset.

Think About:

- What kinds of files contain tabular data in the real world?
- What would you want to know about a dataset the very first time you open it?

Selection: .loc vs .iloc

Once you have a DataFrame, you need to select specific parts of it. Pandas provides two primary accessors for rows:

- `.loc[]` -- selects by **label** (the name of the row or column)
- `.iloc[]` -- selects by **integer position** (the 0-based position number)

Analogy: Imagine a queue at a bank.

- `.iloc[2]` means "give me the 3rd person in line" (position-based -- if people shuffle, you get a different person).
- `.loc['Raj']` means "give me the person named Raj" (label-based -- no matter where Raj stands, you find him by name).

When the index is the default (0, 1, 2, ...), both give the same result. But when the index is something else (names, dates, IDs), they can give very different results. Understanding this distinction is one of the most important skills you will build in this session.

Think About:

- When might you want to select by position rather than by label?
- What could go wrong if you confuse the two?

Boolean Filtering: Asking Questions of Your Data

Just like NumPy boolean masking (which you learned in Session 3.2), Pandas lets you filter rows based on conditions. The syntax will feel very familiar:

```
# NumPy style (you already know this):
arr[arr > 70]

# Pandas style (same idea, but with column names):
df[df['salary'] > 60000]
```

You can also combine multiple conditions and use the `.query()` method for even more readable filtering. The class will cover both approaches.

Think About:

- How would you express "show me all employees in Engineering with a salary above 60000" as a condition?

Sorting: Putting Things in Order

Pandas provides two sorting methods:

- `.sort_values()` -- sort by the data in one or more columns (e.g., sort employees by salary, highest first)
- `.sort_index()` -- sort by the row labels (e.g., sort alphabetically by name if names are the index)

Both support ascending and descending order. You will practice these at the end of the session.

Warm-up Exercises

Try these short exercises before class. Do not worry if you cannot solve them perfectly -- they are meant to get you thinking.

Exercise 1: Dictionary Recall

Create a Python dictionary representing one student's record, then access specific values:

```
student = {
    'name': 'Alice',
    'age': 22,
    'city': 'Mumbai',
    'score': 87.5
}

# Access Alice's score
# Your code here

# Print all the keys (these will become column names in Pandas!)
# Your code here
```

Exercise 2: NumPy Boolean Masking Review

This is exactly the skill that transfers to Pandas filtering:

```
import numpy as np

scores = np.array([45, 78, 92, 34, 88, 67, 55, 91])

# Find all scores above 70
high_scores = # Your code here (hint: boolean masking from Session 3.2)

# Count how many scores are above 70
count = # Your code here
```

```
print(f"High scores: {high_scores}")
print(f"Count: {count}")
```

Exercise 3: Thinking About Labels

Consider this NumPy array representing employee data:

```
import numpy as np

data = np.array([
    [28, 75000],
    [34, 62000],
    [22, 58000],
    [31, 81000]
])
```

Questions to consider:

- What is `data[2, 1]`? Can you tell what that number represents without additional context?
- If someone asked "What is Diana's salary?", how would you find it? You would need to know that Diana is at row 3 and salary is at column 1. Is that convenient?
- What if the array had 50 columns instead of 2? How would you remember which column number corresponds to which attribute?

This is exactly the problem that Pandas solves. In class, you will see how `df.loc['Diana', 'Salary']` makes the same operation self-documenting.

Setup Requirements

Python Version

Python **3.12.x** (the version you have been using since Week 1).

Required Packages

You should already have both of these installed from Week 3. If you do, no action is needed.

Verification Code

Run this in a Jupyter Notebook cell or a Python script before class:

```
import pandas as pd
import numpy as np

print(f"Pandas version: {pd.__version__}")
print(f"NumPy version: {np.__version__}")
print("Ready for Session 4.1!")
```

You should see version numbers printed (Pandas 2.x and NumPy 1.x or 2.x) and the message "Ready for Session 4.1!" with no errors.

If Pandas Is Not Installed

Open your terminal or Anaconda Prompt and run:

```
pip install pandas
```

Or, if you are using Anaconda:

```
conda install pandas
```

Then re-run the verification code above.

Key Vocabulary

These terms will appear throughout the session. Familiarise yourself with them so they are not completely new when you hear them in class.

Term	Definition
Series	A 1D labeled array -- a single column of data with an index attached
DataFrame	A 2D labeled data structure -- a table with named rows and named columns
Index	The row labels in a Series or DataFrame (default: 0, 1, 2, ...)
Column	A single variable or attribute in a DataFrame (e.g., "Name", "Salary")
`.loc`	Label-based selection -- access rows and columns by their names
`.iloc`	Integer position-based selection -- access rows and columns by their 0-based numeric position
Boolean mask	An array of True/False values used to filter rows that meet a condition
`.query()`	A DataFrame method for filtering rows using a string expression
CSV	Comma-Separated Values -- a plain text file format for tabular data
NaN	"Not a Number" -- Pandas representation of missing or unavailable data
`pd.read_csv()`	A Pandas function that reads a CSV file and returns a DataFrame
`.head()` / `.tail()`	Show the first / last 5 rows of a DataFrame
`.info()`	Shows column names, data types, and non-null counts
`.describe()`	Shows statistical summary (mean, std, min, max, etc.) for numeric columns

Term	Definition
`.sort_values()`	Sorts a DataFrame by the values in one or more columns
`.sort_index()`	Sorts a DataFrame by its row index labels

Things to Ponder

Come to class having thought about these questions. You do not need written answers, but engaging with them beforehand will help you follow the session more actively.

About tool choice: You know Python lists, dictionaries, NumPy arrays, and now you are about to learn Pandas. If you had sales data for 50 stores across 12 months -- with store names, city locations, monthly revenue, and employee counts -- which tool would you reach for? What would be difficult about using NumPy alone for this?

About labels vs positions: What is the difference between knowing the POSITION of a row (row number 3) and knowing its LABEL (the row for "Alice")? When might the position change but the label stay the same? When might this distinction cause bugs in your code?

About reproducibility: If a colleague sends you an Excel file with a financial model, how would you verify that every formula is correct? Now imagine the same analysis written as a Python script -- how does that change the verification process? Why did the JPMorgan copy-paste error go unnoticed for so long?

About data exploration: When you receive a brand-new dataset that you have never seen before, what are the first five things you would want to know about it? (Hint: think about size, column names, data types, missing values, and basic statistics.)

Pre-Class Checklist

Before you walk into class, make sure you can check off every item:

- [ ] I have Python 3.12.x installed with Jupyter Notebook (or JupyterLab)
- [ ] I can `import pandas as pd` and `import numpy as np` without errors
- [ ] I have read the Excel Horror Stories section and understand why choosing the right tool matters
- [ ] I understand what tabular data looks like (rows and columns, like a spreadsheet)
- [ ] I know the basic difference between NumPy (same-type numerical arrays) and Pandas (labeled tabular data with mixed types)
- [ ] I know that a Series is a single labeled column and a DataFrame is a labeled table
- [ ] I remember how boolean masking works in NumPy (this transfers directly to Pandas filtering)
- [ ] I have reviewed the vocabulary table above

Looking Ahead

After this session, you will be able to:

- Load any CSV file into Python and start exploring it immediately
- Select exactly the rows and columns you need from a large dataset
- Filter data based on conditions, just like NumPy boolean masking but with named columns
- Sort your results by any column or combination of columns
- Understand the output of `.info()` and `.describe()` -- the two methods every data scientist runs first on every new dataset

This session sets the foundation for **Session 4.2**, where you will learn data wrangling: cleaning messy data, merging multiple datasets, and using GroupBy for powerful aggregations. Everything you learn today is a prerequisite for everything that follows.

See you in class!

Vishlesan i-Hub IIT Patna x Masai School